

# Kluczowe elementy rozwiązania – uzupełnienie

---

Projekt: Optymalna trasa turystyczna

---

## 1. Cel projektu

---

Celem projektu jest implementacja aplikacji webowej umożliwiającej wyznaczenie optymalnej trasy turystycznej z uwzględnieniem:

- czasu oraz miejsca rozpoczęcia wycieczki,
- miejsc wskazanych przez użytkownika,
- czasu pobytu w miejscach,
- godzin otwarcia,
- opcjonalnej godziny zakończenia wycieczki określonej przez użytkownika
- wybranego środka transportu.

Aplikacja umożliwia planowanie wycieczki możliwej do realizacji:

- pieszo,
  - samochodem,
  - komunikacją miejską.
- 

## 2. Problem optymalizacji

---

Wyznaczanie optymalnej trasy jest problemem wielokryterialnym.

Algorytm powinien:

- minimalizować czas podróży,
  - maksymalizować czas spędzony w atrakcjach,
  - uwzględniać godziny otwarcia miejsc,
  - spełniać ograniczenia czasowe użytkownika,
  - ograniczać liczbę pominiętych atrakcji.
- 

## 3. Źródła pozyskiwanych danych

---

Aplikacja wykorzystuje zewnętrzne API Google do pobierania danych niezbędnych do wyznaczania trasy.

### Google Maps API

Wykorzystywane do:

- pobierania współrzędnych geograficznych miejsc,

- wyszukiwania atrakcji i restauracji,

## Google Distance Matrix API

Wykorzystywane do:

- obliczania czasów przejazdu pomiędzy punktami,
- wyznaczania odległości,
- uwzględniania wybranego środka transportu,
- budowy macierzy czasów przejazdu wykorzystywanej przez algorytm optymalizacji.

## Zakres pozyskiwanych danych

Podczas wyszukiwania miejsc system korzysta z danych udostępnianych przez Google Maps API, takich jak:

- nazwa miejsca,
- adres,
- współrzędne geograficzne.

Po wybraniu punktów przez użytkownika system pobiera dodatkowo:

- godziny otwarcia,
- czasy przejazdu pomiędzy punktami,
- odległości pomiędzy lokalizacjami.

Dane te wykorzystywane są do budowy macierzy odległości oraz działania algorytmu optymalizacji trasy. Macierz odległości budowana jest wyłącznie dla punktów wskazanych przez użytkownika, co pozwala ograniczyć liczbę zapytań do API oraz zmniejszyć koszty wykorzystania usług Google.

---

## 4. Architektura aplikacji

---

Aplikacja została podzielona na frontend oraz backend.

### Frontend

Frontend odpowiada za:

- interakcję z użytkownikiem,
- pobieranie parametrów wycieczki,
- wizualizację trasy na mapie.

### Technologie

- React,
- React Google maps.

---

### Backend

Backend odpowiada za:

- komunikację z Google API,
- pobieranie danych o miejscach,
- działanie algorytmu optymalizacji,
- generowanie końcowej trasy.

Technologie

- Python,
- FastAPI,
- NetworkX.

---

## 5. Algorytm optymalizacji

---

W projekcie zastosowano algorytm zachłanny rozszerzony o obsługę ograniczeń czasowych oraz godzin otwarcia atrakcji.

### Schemat działania

1. Rozpoczęcie wycieczki w punkcie startowym wskazanym przez użytkownika.
2. Wyznaczenie listy nieodwiedzonych atrakcji.
3. Odrzucenie atrakcji, które:
  - są już zamknięte,
  - nie mogą zostać odwiedzone przed godziną zamknięcia,
  - spowodowałyby przekroczenie czasu zakończenia wycieczki.
4. Obliczenie kosztu przejścia do każdej możliwej atrakcji.
5. Uwzględnienie ograniczeń użytkownika:
  - czasu pobytu,
  - maksymalnego czasu wycieczki.
6. Wybór atrakcji o najniższym koszcie.
7. Aktualizacja:
  - aktualnej pozycji użytkownika,
  - bieżącego czasu wycieczki,
  - listy odwiedzonych miejsc.
8. Powtarzanie procesu aż do:
  - odwiedzenia wszystkich atrakcji,
  - braku możliwych do odwiedzenia miejsc,
  - osiągnięcia limitu czasu wycieczki.

---

## 6. Funkcja kosztu

---

$$C(i,j) = a * t(i,j)$$

$$\begin{aligned} &+ b * U(j) \\ &- c * A(j) \\ &+ d * P(j) \end{aligned}$$

Gdzie:

- $t(i, j)$  — czas przejazdu pomiędzy punktami,
- $U(j)$  — współczynnik pilności wynikający z godziny zamknięcia atrakcji,
- $A(j)$  — współczynnik atrakcyjności miejsca,
- $P(j)$  — kara za niedopasowanie czasowe względem preferencji użytkownika,
- $a, b, c, d$  — współczynniki wag.

---

## Kara za niedopasowanie czasowe

Wartość  $P(j)$  określa stopień niedopasowania czasu odwiedzenia atrakcji do preferencji użytkownika.

Przykładowo:

$$P(j) = 0$$

jeżeli atrakcja jest odwiedzana w preferowanym przedziale czasowym użytkownika,

oraz:

$$P(j) > 0$$

jeżeli:

- użytkownik przybędzie za wcześnie,
- użytkownik przybędzie później niż preferowany czas,
- konieczne będzie długie oczekiwanie na otwarcie atrakcji.

---

## Ograniczenia czasowe

Atrakcja może zostać wybrana wyłącznie wtedy, gdy spełnione są następujące warunki:

Warunek godzin otwarcia

$$\text{arrival}(j) + \text{stay}(j) \leq \text{closing}(j)$$

Warunek zakończenia wycieczki

```
currentTime + t(i,j) + stay(j) <= T_end
```

Gdzie:

- `arrival(j)` — przewidywany czas przybycia,
- `stay(j)` — przewidywany czas pobytu,
- `closing(j)` — godzina zamknięcia atrakcji,
- `T_end` — maksymalny czas zakończenia wycieczki.

---

## Charakterystyka algorytmu

Algorytm zachłanny wybiera w każdym kroku lokalnie najlepszą atrakcję na podstawie funkcji kosztu. Rozwiązanie pozwala uzyskać trasę możliwą do realizacji w krótkim czasie obliczeń, jednak nie gwarantuje znalezienia rozwiązania globalnie optymalnego.

---

## 7. Uwzględnianie ograniczeń użytkownika

Algorytm podczas wyboru kolejnych punktów sprawdza:

- czy miejsce jest otwarte,
- czy użytkownik zdąży odwiedzić punkt przed zamknięciem,
- czy możliwe jest dotrzymanie godziny zakończenia wycieczki,
- czy punkt mieści się w preferowanym przedziale czasowym użytkownika.

Punkty niespełniające ograniczeń krytycznych są odrzucane podczas działania algorytmu. Dotyczy to sytuacji, gdy:

- miejsce jest zamknięte,
- użytkownik nie zdąży zakończyć wizyty przed zamknięciem,
- odwiedzenie punktu spowodowałoby przekroczenie czasu zakończenia wycieczki.

W przypadku naruszenia preferowanego przedziału czasowego punkt nie jest odrzucany, lecz otrzymuje dodatkową karę w funkcji kosztu. Dzięki temu algorytm może nadal wybrać taką atrakcję, jeżeli będzie ona korzystna względem pozostałych dostępnych opcji.

---

## 8. Kryteria oceny trasy

Ocena jakości wyznaczonej trasy odbywa się na podstawie:

- całkowitego czasu podróży,
- całkowitego czasu pobytu w atrakcjach,
- liczby odwiedzonych miejsc,
- liczby pominiętych atrakcji,
- liczby naruszeń ograniczeń czasowych,

- długości trasy,
  - czasu działania algorytmu.
- 

## 9. Testy i scenariusze testowe

---

Testy mają na celu ocenę:

- jakości wyznaczanych tras,
  - wydajności algorytmu,
  - praktycznej przydatności rozwiązania.
- 

### Scenariusz 1 – Mała liczba punktów

- 5–8 atrakcji,
- brak ograniczeń czasowych.

Cel:

- sprawdzenie poprawności działania algorytmu.

Dla małych przypadków planowane jest porównanie wyników algorytmu zachłannego z algorytmem brute force, który sprawdza wszystkie możliwe permutacje tras i pozwala wyznaczyć rozwiązanie optymalne.

Pozwoli to ocenić jakość rozwiązania heurystycznego.

---

### Scenariusz 2 – Ograniczone godziny otwarcia

- część atrakcji zamykana wcześniej,
- konflikty czasowe pomiędzy punktami.

Cel:

- sprawdzenie poprawności uwzględniania ograniczeń czasowych.
- 

### Scenariusz 3 – Ograniczony czas wycieczki

- użytkownik posiada limit czasu,
- liczba atrakcji większa niż możliwa do odwiedzenia.

Cel:

- ocena wyboru najbardziej opłacalnych punktów.
- 

### Scenariusz 4 – Duża liczba punktów

- 30–50 atrakcji.

Cel:

- pomiar czasu działania algorytmu,
  - ocena skalowalności rozwiązania.
- 

## Scenariusz 5 – Różne środki transportu

- pieszo,
- samochód,
- komunikacja miejska.

Cel:

- porównanie jakości oraz długości tras.
- 

## Scenariusz 6 – Konflikty preferencji użytkownika

- wymuszone godziny odwiedzin,
- obowiązkowe punkty,
- ograniczony czas wycieczki.

Cel:

- sprawdzenie działania algorytmu przy wielu jednoczesnych ograniczeniach.
- 

# 10. Wyniki działania aplikacji

---

Wynikiem działania systemu jest:

- wyznaczona trasa wycieczki,
- wizualizacja trasy na mapie,
- kolejność odwiedzanych punktów,
- całkowity czas wycieczki,
- szacowany czas przejazdu oraz pobytu.